



Université de Sherbrooke

SQL

Expressions simples

SQL_03

Christina KHNAISSER (christina.khnaisser@usherbrooke.ca)

Luc LAVOIE (luc.lavoie@usherbrooke.ca)

Ameni SOUID (ameni.Souid@usherbrooke.ca)

(les auteurs sont cités en ordre alphabétique nominal)

—

Scriptorum/Scriptorum/SQL_03a-Expressions-simples, version 1.0.1.a, en date du 2026-01-29

— document en cours de révision, ne pas citer à l'externe —

Sommaire

Ce module présente une introduction à la définition de requêtes en SQL. On y décrit une version simplifiée de l'instruction SQL couvrant les opérations de jointure-restriction-projection-extension. On y indique également la manière et l'importance de faire les renommages appropriés. Finalement

Mise en garde

Le présent document est en cours d'élaboration ; en conséquence, il est incomplet et peut contenir des erreurs.

Historique

diffusion	resp.	description
2026-01-24	CK	Récupération de notes diverses SQL03b.

Table des matières

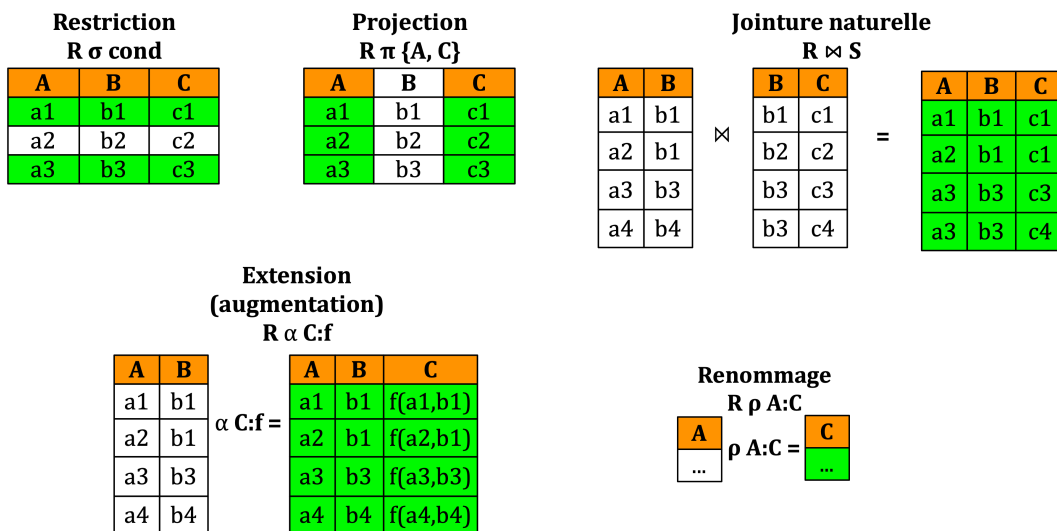
Introduction.....	4
1. Présentation.....	4
2. Projection-Extension-Renommage.....	4
2.1. Opérateur CASE.....	8
2.2. Fonctions d'aggrégations.....	10
3. Restriction.....	12
4. Jointure.....	17
4.1. Produit.....	18
4.2. Jointure naturelle.....	19
4.3. Jointure qualifiée.....	20
4.4. Jointure externe.....	21
5. Opérations complémentaires.....	23
Conclusion.....	28
Références.....	29
Définitions.....	30

Introduction

Dans un premier temps, nous présenterons une grammaire simplifiée, sous-ensemble propre de la grammaire de la norme ISO 9075:2016, avec, au besoin, certaines particularités de PostgreSQL.

La séquence d'opérations jointure-restriction-projection-extension contribue en pratique à une proportion importante des requêtes, dans la mesure d'un renommage occasionnel des termes. En réunissant cette séquence dans une même instruction, les concepteurs du langage SQL voulaient donner une allure procédurale au langage, jugeant qu'elle faciliterait l'adhésion des programmeurs; offrir un raccourci notationnel aux programmeurs; faciliter l'optimisation conjointe des opérations. Les opérateurs ensemblistes (union, différence, intersection) seront intégrés par la suite, d'une façon moins commode. Au fil du temps, plusieurs opérateurs (ou pseudo-opérateurs) seront ajoutés – certains d'entre eux seront couverts dans le module suivant.

1. Présentation



Une requête n'est pas autre chose qu'une expression SELECT.

```
requête ::=
  [ Contexte ]
  SELECT opMode { * | projection-extension }
  FROM listeDeJointures
  [ Restriction ]
  [ Groupement ]
  [ OpComplémentaire ]
  [ Ordonnancement ]
  [ Divers ]
```

2. Projection-Extension-Rennommage

Définition de l'entête du résultat.

```
SELECT opMode { * | projection-extension }
FROM listeDeJointures

opMode ::=
  [ DISTINCT | ALL ]

projection-extension ::=
```

```
{ expression [ [ AS ] nomColonne ] ... , }  
listeDeJointures ::=  
  nomTable
```

Le mode *opMode* indique si l'expression retourne une relation (DISTINCT) ou une collection (ALL).

L'étoile indique que tous les attributs de l'expression sont retenus (sans projection).

La clause projection-extension permet de ne retenir que les expressions désirées et de les nommer. Les attributs y sont limités aux seuls attributs obtenus par la jointure de la clause FROM qui suit. Lorsqu'une expression se réduit à un identificateur, il s'agit en fait simplement d'une projection de la théorie relationnelle.

Le mot AS est superfétatoire, mais son utilisation est recommandée. L'identifiant alias permet de nommer l'expression en même temps

Exemple — Université

- Quels sont les types d'évaluation ?

- Quels sont les matricules des personnes étudiantes admises ?

Clarification: on suppose ici que Etudiant donne la liste des personnes étudiantes ayant été admises.

2.1. Opérateur CASE

C'est un opérateur de choix.

C'est-à-dire qu'il permet de choisir une valeur sur la base d'une catégorisation établie à l'aide d'une condition (simple case) ou d'une énumération de valeurs (searched case). Il en existe donc deux formes
<case expression> ::= <simple case> | <searched case> Dans tous les cas, l'opérateur retourne une valeur.

```
<simple case> ::=  
    CASE WHEN <condition> THEN <result>  
    [ ELSE <result> ]  
    END  
  
<result> ::=  
    <expression> | NULL
```

Tous les résultats doivent appartenir au même type. Si la valeur de l'expression est présente dans plus d'une liste, la première occurrence est choisie. En l'absence de clause ELSE, si la valeur de l'expression n'apparaît dans aucune liste, la « valeur » NULL est retournée!

Exemple — Université

- Quelles sont les personnes étudiantes inscrites ?
Donner le matricule et le nom du département.

Clarification: on suppose ici qu'une personne étudiante n'est inscrite qu'à partir du moment où elle a un résultat d'évaluation.

- Étendre la table résultat pour ajouter :
la cote 'A' si la note est au moins de 90 et
la cote 'B' si la note au moins de 70 (mais inférieure à 90).

2.2. Fonctions d'aggrégations

- AVG(col)
- COUNT(col)
- MAX(col)
- MIN(col)
- SUM(col)
- MEDIAN(col)

Plusieurs autres disponibles avec PostgreSQL, dont :

- string_agg
- xml_agg
- json_agg
- array_agg
- range_agg
- bit_or, bit_and

de très nombreuses fonctions statistiques...

Quand mettre ALL ou DISTINCT ?
Que se passe-t-il s'il n'y a aucun tuple ?
Comment les NULL sont-ils traités ?
Un cas spécial : COUNT(*)

Exemple— Université

- Combien y-a-t'il d'activités offertes ?

- Calculer la moyenne, le minimum et le maximum des résultats;

3. Restriction

Définition de l'ensemble de tuples à retenir.

```
requête ::=  
  [ Contexte ]  
  SELECT opMode { * | projection-extension }  
  FROM listeDeJointures  
  WHERE condition  
  [ Groupement ]  
  [ OpComplémentaire ]  
  [ Ordonnancement ]  
  [ Divers ]
```

Les attributs qui peuvent être utilisés dans la condition sont limités aux seuls attributs obtenus par la jointure de la clause FROM qui précède.

Exemple — Université

- Quelles sont les activités offertes au département d'informatique ?

- Quelles sont les personnes étudiantes inscrites à l'activité IFT187 ?

- Quelles sont les personnes étudiantes inscrites à une activité à l'automne ?

- Calculer la moyenne, le minimum et le maximum des résultats des examens (IN, FI) de l'activité IFT187;

- Quelles sont les personnes étudiantes ayant un résultat d'examen entre 70 et 100 ?

4. Jointure

```
SELECT opMode { * | projection-extension }
FROM listeDeJointures

listeDeJointures ::=
  denotationTable [ [AS] alias] [ jointure ... ]

denotationTable ::=
  nomTable | ( requête )

jointure ::=
  produit | jointure_naturelle | jointure_qualifiée | jointure_externe
```

Une *listeDeJointures* permet de faire plusieurs jointures à la suite. Chaque terme est représenté par une *denotationTable*, en pratique un nom de table ou une requête entre parenthèses.

Le mot *AS* est superfétatoire, mais son utilisation est recommandée. L'identifiant *alias* permet de (re)nommer la *denotationTable*.

Exemple — Université

- Quels sont les étudiants ayant obtenu une note supérieure ou égale à 70 ? afficher le matricule, le sigle de l'activité, le titre de l'activité et la note.

4.1. Produit

```
produit ::=  
  {CROSS JOIN | , } denotationTable [ [ AS ] alias ]
```

Exemple — Université

- Afficher toutes les combinaisons possibles entre les étudiants et les activités. Pour chaque combinaison, afficher le matricule et le nom de l'étudiant ainsi que le sigle et le titre de l'activité.

4.2. Jointure naturelle

```
jointure_naturelle ::=  
  NATURAL [ INNER ] JOIN denotationTable [ [ AS ] alias ]
```

La jointure naturelle opère sur tous les attributs de même nom et élimine (automatiquement) la redondance des attributs et leurs résultats.

Exemple — Université

- Produire la liste des étudiants pour lesquels un résultat existe, avec leur matricule, leur nom, l'activité et la note.

4.3. Jointure qualifiée

```
jointure_qualifiée ::=  
  [ INNER ] JOIN denotationTable [ [ AS ] alias ] qualification  
  
qualification ::=  
  ON conditionJointure | USING ( listeNomCol )
```

- ON $a_1=b_1$ AND ... AND $a_n=b_n$ lorsque les identifiants des attributs de jointure ne sont pas les mêmes (on désigne souvent cette jointure comme étant conditionnelle).
- USING ($a_1, \dots, a_n$) lorsque les identifiants des attributs de jointure sont les mêmes dans les deux tables (on désigne souvent cette jointure comme étant restreinte).

Exemple — Université

- Produire le bulletin d'Éliane. Donner le matricule, le nom, le sigle de l'activité, le nom de l'activité, le trimestre, le type d'évaluation et la note.

4.4. Jointure externe

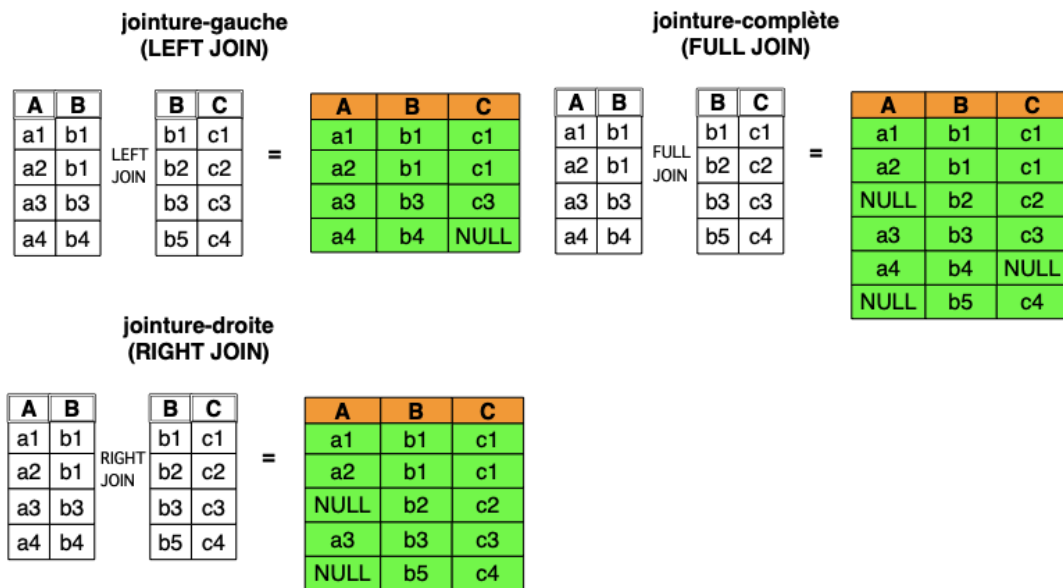


Figure 1. Illustration des jointures externes

```
jointure_externe ::=
  jExterne JOIN denotationTable [ [ AS ] alias ] qualification

jExterne ::=
  LEFT [OUTER] | RIGHT [OUTER] | FULL [OUTER]
```

Exemple — Université

- Quelles sont les personnes étudiantes n'ayant aucune évaluation à ce jour ?
Donner le matricule et le nom de la personne.

- Quelles sont les activités n'ayant aucune évaluation à ce jour ?
Donner le sigle et le titre de l'activité.

5. Opérations complémentaires

```
OpComplémentaire ::=  
  INTERSECT opMode requête  
  | UNION    opMode requête  
  | EXCEPT opMode requête
```

Les opérateurs ensemblistes.

Le mode *opMode* par défaut (DISTINCT) agit comme un ensemble. C'est-à-dire qu'il n'y aura pas de doublons dans les résultats.

Exemple — Université

- Quelles sont les activités ayant au moins une évaluation à ce jour ?

- Quelles sont les activités n'ayant aucune évaluation à ce jour ?

- Quelles sont les personnes étudiantes qui ne sont inscrites à aucune activité?

- Quelle est la moyenne des notes pour IFT187 et IFT159 ?

UNIVERSITÉ

Étudiant clé (matricule)

matricule	nom	adresse
15113150	Paul	>Δ ^ς σ ^ς ᵇ
15112354	Éliane	Blanc-Sablon
15113870	Mohamed	Tadoussac
15110132	Sergeï	Chandler

Activité clé (sigle)

sigle	titre
IFT159	Analyse et programmation
IFT187	Éléments de bases de données
IMN117	Acquisition des médias numériques
IGE401	Gestion de projets
GMQ103	Géopositionnement

TypeEvaluation clé(code)

code	description
IN	Examen intra
FI	Examen final
TP	Travail pratique
PR	Projet

Résultat clé(matricule, te, activite, trimestre)

matricule	TE	activité	trimestre	note
15113150	TP	IFT187	20132	80
15112354	FI	IFT187	20122	78
15113150	TP	IFT159	20132	75
15112354	FI	GMQ103	20122	85
15110132	IN	IMN117	20123	90
15110132	IN	IFT187	20133	45
15112354	FI	IFT159	20123	52

Figure 2. Exemple — Université

Conclusion

La séquence d'opérations jointure-restriction-projection-extension contribue en pratique à une proportion importante des requêtes, dans la mesure d'un renommage occasionnel des termes. En réunissant cette séquence dans une même instruction, les concepteurs du langage SQL voulaient

- donner une allure procédurale au langage, jugeant qu'elle faciliterait l'adhésion des programmeurs;
- offrir un raccourci notationnel aux programmeurs;
- faciliter l'optimisation conjointe des opérations.

Les opérateurs ensemblistes (union, différence, intersection) ont été intégrés par la suite, d'une façon moins commode.

Au fil du temps, plusieurs opérateurs (ou pseudo-opérateurs) seront ajoutés – certains d'entre eux seront couverts dans le module suivant.

Références

[Elmasri2016]

Ramez ELMASRI et Shamkant B. NAVATHE;
Fundamentals of database systems;
7th Edition, Pearson, Hoboken (NJ, US), 2016;
ISBN 978-0-13-397077-7.

[Date2015a]

Chris J. DATE;
SQL and relational theory: how to write accurate SQL code;
O'Reilly Media, 2015;
ISBN 978-1-4919-4117-1.

PG

[fr] <https://docs.postgresql.fr/18/index.html> (2026-01-05)

[en] <https://www.postgresql.org/docs/18/index.html> (2026-01-05)

Définitions

SQL (*Structure Query Language*)

Langage de programmation axiomatique fondé sur un modèle inspiré de la théorie relationnelle proposée par E. F. Codd.

[Normes applicables : ISO 9075:2016, ISO 9075:2023]

Produit le 2026-09-29 00:12:55 UTC



Université de Sherbrooke